

Week 6: Ensembles

CSCI 447: Machine Learning— Fall 2026



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

Rule Pruning

- Two approaches to pruning
 - Prune whole rules
 - Prune predicates within rules

$$X \wedge \cancel{X} \wedge Z \Rightarrow c$$

- Note that pruning predicates *subsumes* pruning rules
- Sort the rules by coverage
- Resulting pruning strategy, similar to decision trees is
Incremental Reduced Error Pruning (IREP)



Pruning

Algorithm 9.4 Incremental Reduced Error Pruning (IREP)

```
1: function IREP(Pos, Neg)
2:   ruleSet  $\leftarrow \{\}$ 
3:   while Pos  $\neq \emptyset$  do
4:     SPLIT(Pos, Neg) into (GrowPos, GrowNeg) and (PrunePos, PruneNeg)
5:     Rule  $\leftarrow$  GrowRule(GrowPos, GrowNeg)
6:     Rule  $\leftarrow$  PruneRule(PrunePos, PruneNeg)
7:     if Perf(Rule, (PrunePos, PruneNeg))  $< \theta$  then  $\triangleright$  Best rule worse than chance
8:       return ruleSet
9:     else
10:      RuleSet  $\leftarrow$  RuleSet + Rule
11:      (Pos, Neg)  $\leftarrow$  (Pos, Neg) – Covered(Rule)
12:    end if
13:  end while
14:  return ruleSet
15: end function
```

Grow using 2/3 of the data; Prune using 1/3 of the data



The PruneRule Function

- Let $P = |PrunePos|$
- Let $N = |PruneNeg|$
- For the current rule, let p be the number of examples in $PrunePos$ covered by the rule, and n be the number of examples in $PruneNeg$ covered by the rule.

- Define

$$v(Rule, PrunePos, PruneNeg) = \frac{p + N - n}{P + N}$$

- Choose the predicate the prune that maximizes v .



RIPPER

- “Repeated Incremental Pruning to Produce Error Reduction”
- Let $RuleSet = \{Rule_1, \dots, Rule_k\}$
- Two options:
 1. Replace Rule: Total error
 2. Revise Rule: Modify predicates
- Make decisions based on Minimum Description Length

$$\mathcal{L}(\mathcal{D}) = \min_{H \in \mathcal{H}} [\mathcal{L}(H) + \mathcal{L}(\mathcal{D}|H)]$$



Minimum Description Length

An information theoretic approach

$$\begin{aligned} H_{map} &= \operatorname{argmax}_{H \in \mathcal{H}} P(\mathcal{D}|H)P(H) \\ &= \operatorname{argmax}_{H \in \mathcal{H}} \log P(\mathcal{D}|H) + \log P(H) \\ &= \operatorname{argmin}_{H \in \mathcal{H}} -\log P(\mathcal{D}|H) - \log P(H) \end{aligned}$$

Note: $\mathcal{L}(\mathcal{D}|H) = -\log P(\mathcal{D}|H)$ and $\mathcal{L}(H) = -\log P(H)$

So

$$H_{map} = \operatorname{argmin}_{H \in \mathcal{H}} \mathcal{L}(\mathcal{D}) = \operatorname{argmin}_{H \in \mathcal{H}} \mathcal{L}(\mathcal{D}|H) + \mathcal{L}(H)$$



RIPPER Algorithm

```
1: function RIPPER(Pos, Neg)
2:   ruleSet  $\leftarrow$  BuildRuleSet(Pos, Neg)
3:   repeat
4:     ruleSet  $\leftarrow$  OptimizeRuleSet(ruleSet, Pos, Neg)
5:   until k times
6:   return ruleSet
7: end function
```



Building the Rule Set

```
8: function BUILDRULESET(Pos, Neg)
9:   ruleSet  $\leftarrow$  {}
10:  DL  $\leftarrow$  DescriptionLength(ruleSet, Pos, Neg)
11:  while Pos  $\neq$   $\emptyset$  do
12:    SPLIT(Pos, Neg) into (GrowPos, GrowNeg) & (PrunePos, PruneNeg)
13:    Rule  $\leftarrow$  GrowRule(GrowPos, GrowNeg)
14:    Rule  $\leftarrow$  PruneRule(Rule, PrunePos, PruneNeg)
15:    RuleSet  $\leftarrow$  RuleSet + Rule
16:    if DescriptionLength(ruleSet, Pos, Neg) > DL + 64 then
17:      for each rule R in ruleSet do
18:        if DescriptionLength(ruleSet - {R}, Pos, Neg) < DL then
19:          DL  $\leftarrow$  DescriptionLength(ruleSet, Pos, Neg)
20:        end if
21:      end for
22:      return ruleSet
23:    end if
24:  end while
25: end function
```



Optimizing the Rule Set

```
26: function OPTIMIZERULESET(ruleSet, Pos, Neg)
27:   for each rule R in ruleSet do
28:     Delete R from ruleSet
29:     UPos  $\leftarrow$  examples in Pos not covered by ruleSet
30:     UNeg  $\leftarrow$  examples in Neg not covered by ruleSet
31:     Split (UPos, UNeg) into (GrowPos, GrowNeg) & (PrunePos, PruneNeg)
32:     RepRule  $\leftarrow$  GrowRule(GrowPos, GrowNeg)
33:     RepRule  $\leftarrow$  PruneRule(RepRule, PrunePos, PruneNeg)
34:     RevRule  $\leftarrow$  GrowRule(GrowPos, GrowNeg, R)
35:     RevRule  $\leftarrow$  PruneRule(RevRule, PrunePos, PruneNeg)
36:     Choose better of RepRule and RevRule and add to ruleSet
37:   end for
38:   return ruleSet
39: end function
```



Ensembles – Motivation

- It is difficult to determine the appropriate inductive bias for a learning method with respect to a target data set.
- Effect: Susceptibility to either overfitting (high bias/low variance) or underfitting (low bias/high variance).
- Approach:
 - Train multiple base learners (models)
 - Combine predictions from base learners
- Intended Result: Simultaneous reduction of bias and variance
- CAUTION!
 - The bias-variance tradeoff still applies
 - The "No Free Lunch" Theorems still apply



Problem

- Given:
 - Training data set $\mathbf{X}$ for supervised learning.
 - $\mathbf{X}$ drawn from common instance space $\mathcal{X}$.
 - Collection of inductive learning algorithms.
- Hypotheses produced by applying inducers to $s(\mathbf{X})$.
 - $s: \mathbf{X} \rightarrow \mathbf{X}'$ (sampling, transformation, partitioning, etc.).
 - Consider hypotheses as $\mathbf{H}$ definitions of prediction algorithms (“base learners”).
- Return: A new prediction algorithm (not necessarily $\in \mathbf{H}$) for $\mathbf{x} \in \mathbf{X}$ that combines outputs from collection of prediction algorithms.



Ensemble Methods

- Bagging
 - Short for “bootstrap aggregation”
 - Generate multiple sets of bootstrap samples and train on each set
 - Apply basic aggregation
- Boosting
 - A sequential process for training base learners
 - Each learner is trained based on the performance of previous learners
 - Apply a weighted aggregation scheme
- Stacking
 - Train different base learners (i.e., use different algorithms)
 - Train an aggregator to determine how best to combine



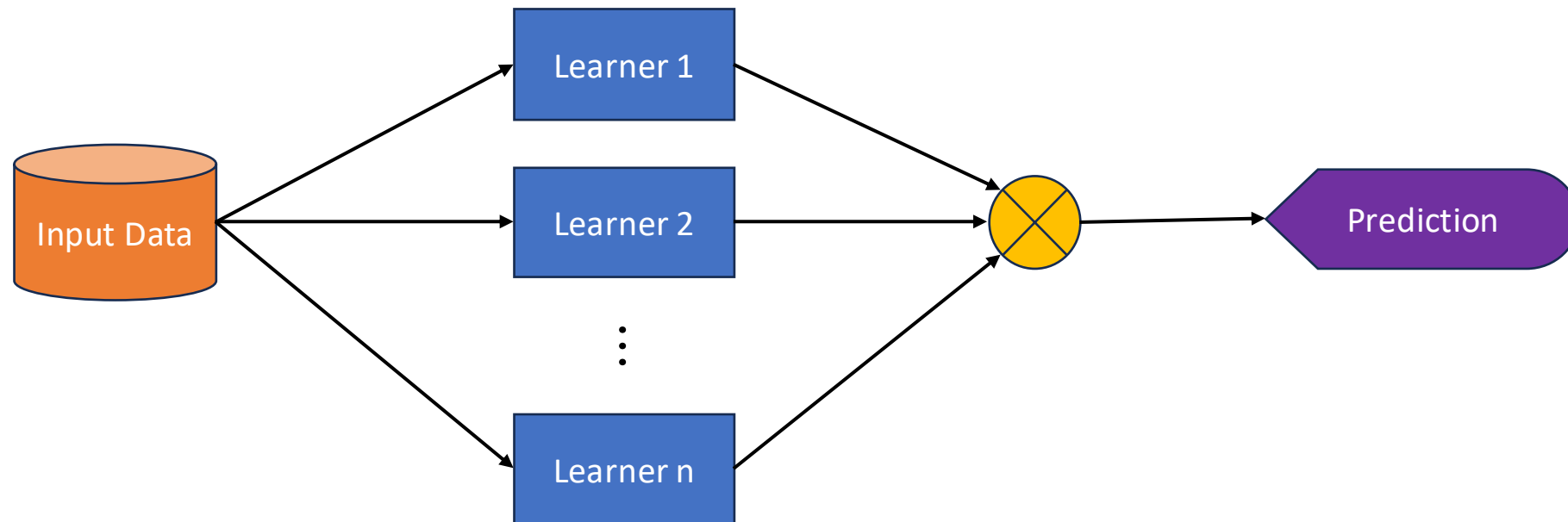
Critical Factors

- Accuracy
 - The expectation is that the base learners be trained to be as accurate as possible
 - Note that some ensemble methods only require “weak” learners, i.e., learners capable of performing only lightly better than chance
 - Accuracy comes from the aggregation process
- Diversity
 - The base learners need to have knowledge about different aspects of the task
 - Can be accomplished in a variety of ways
 - Diverse training sets
 - Reweighting of the same training set
 - Applying multiple methods with different inductive biases



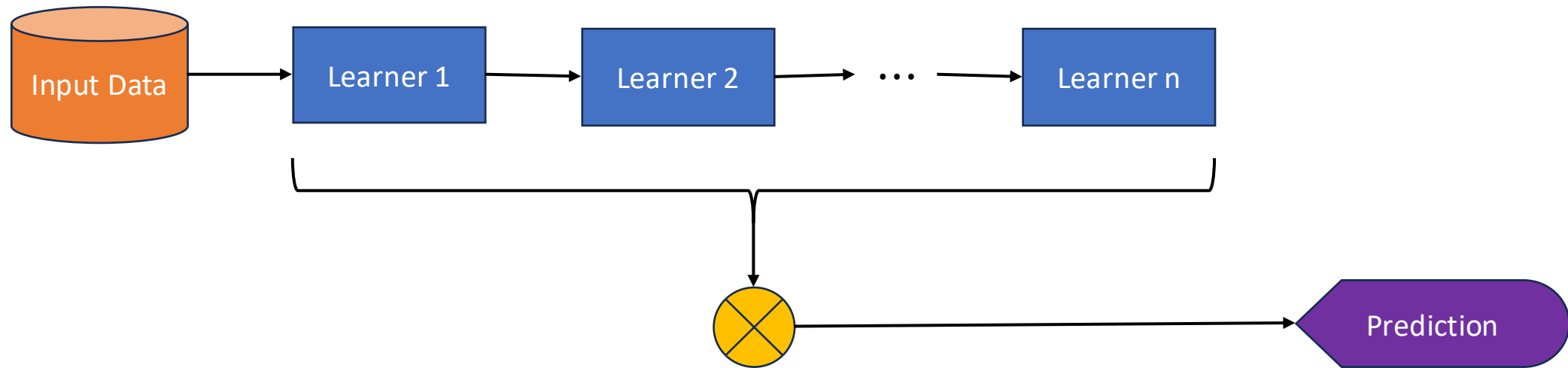
General Approaches: Parallel

- Train base learners independently
- Combine the results through simple aggregation



General Approaches: Sequential

- Train base learners iteratively
- Focus of training is correcting errors from previous learners



Diversity

- The goal is to train a set of base learners negatively correlated with one another
- Ensemble diversity can be computed “pairwise” or “non-pairwise”
- Let $\mathcal{D}$ denote the ensemble of base learners
- A pairwise diversity measure (Yule’s Q Statistic) – Want as close to -1 as possible
 - Let $y_{j,i}$ be an indicator of whether learner i recognizes data item $\mathbf{x}_j$ correctly, and let N^{ab} denote the number of times $y_{j,i} = a$ and $y_{j,k} = b$.
 - $Q_{i,k} = \frac{N^{11}N^{00} - N^{01}N^{10}}{N^{11}N^{00} + N^{01}N^{10}}$ and $\bar{Q} = \frac{1}{L(L-1)} \sum_{i=1}^{L-1} \sum_{k=i+1}^L Q_{i,k}$.
- A non-pairwise diversity measure (Entropy) – Want as close to $+1$ as possible
 - Let $\ell(\mathbf{x}_j)$ denote the number of learners that correctly classify $\mathbf{x}_j$.
 - $E(\mathcal{D}) = \frac{1}{N} \sum_{j=1}^N \frac{1}{(L - \lfloor L/2 \rfloor)} \min\{\ell(\mathbf{x}_j), L - \ell(\mathbf{x}_j)\}$



Combining Learners

- In ensemble learning, the results of predictions from the base learners need to be combined to generate a final prediction.
- Such combiners are often referred to as “aggregators.”
- Combining/aggregating predictions varies by task (classification/regression).
- The type of ensemble generally dictates the method of aggregation:
 - Bagging/boosting: Apply static combination rule after training
 - Stacking: Train an aggregation model based on performance of base learners



Majority Vote

- Arguably one of the “simplest” methods.
- Applied specifically to classification problems.
- Given a set of base learners $\mathcal{D}$, let $d_{t,c} \in \{0,1\}$ denote whether classifier t indicated class c .
- Generate prediction as follows:

$$class = \operatorname{argmax}_{c \in \mathcal{C}} \sum_{t=1}^{|\mathcal{D}|} d_{t,c}$$



Weighted Majority Vote

- Adjust majority vote based on performance of base learners.
- Applied specifically to classification problems.
- Given a set of base learners $\mathcal{D}$, let $d_{t,c} \in \{0,1\}$ denote whether classifier t indicated class c .
- Let $w_t = L_t / \sum_{t'=1}^{|\mathcal{D}|} L'_{t'}$, where L_t is the estimated accuracy of learner t .
- Generate prediction as follows:

$$class = \operatorname{argmax}_{c \in \mathcal{C}} \sum_{t=1}^{|\mathcal{D}|} w_t d_{t,c}$$



Weighted Majority Vote

Algorithm 12.1 Weighted Majority Ensemble

```
1: function COMBINEWEIGHTEDMAJORITY( $\mathcal{D}$ ,  $\mathcal{L}$ )
2:    $n \leftarrow |\mathcal{L}|$  ▷ the number of models in  $\mathcal{L}$ 
3:    $m \leftarrow |\mathcal{D}|$  ▷ the number of examples in the training set
4:   for  $i \leftarrow 1, n$  do
5:      $\ell_i \leftarrow \text{Train}(\mathcal{L}_i, \mathcal{D})$ 
6:      $w_i \leftarrow 1$  ▷ initialize weight to unity
7:   end for
8:   for  $j \leftarrow 1, m$  do
9:      $q_0 \leftarrow 0$ ;  $q_1 \leftarrow 0$ 
10:    for  $i \leftarrow 1, n$  do
11:      if  $\ell_i(\mathbf{x}_j) = 0$  then
12:         $q_0 \leftarrow q_0 + w_i$  ▷ vote for class 0
13:      end if
14:      if  $\ell_i(\mathbf{x}_j) = 1$  then
15:         $q_1 \leftarrow q_1 + w_i$  ▷ vote for class 1
16:      end if
17:    end for
18:     $\ell_i(\mathbf{x}_j) \leftarrow (q_0 > q_1) ? 0 : ((q_0 < q_1) ? 1 : \text{rand}(0, 1))$ 
19:    for  $i \leftarrow 1, n$  do
20:      if  $\ell_i(\mathbf{x}_j) \neq \text{class}'(\mathbf{x}_j)$  then ▷  $\text{class}'(\mathbf{x}_j)$  is the true class
21:         $w_i \leftarrow \beta w_i$  ▷  $\beta \in (0, 1)$  is a penalty factor
22:      else
23:         $w_i \leftarrow w_i / \beta$ 
24:      end if
25:    end for
26:  end for
27:  return MakePredictor( $\mathcal{L}$ ,  $\mathbf{w}$ )
28: end function
```



Simple Aggregation – Regression

- The process for aggregation in regression is similar to that of classification.
- Replace vote with finding the (weighted) mean (or median).
- Let $\hat{y}_t$ denote the prediction/response from the t th base learner
- Let $w_t = (1/L_t) / \sum_{t'=1}^{|\mathcal{D}|} (1/L_{t'})$, where L_t is the estimated error of learner t .
- For unweighted:

$$resp = \frac{1}{|\mathcal{D}|} \sum_{t=1}^{|\mathcal{D}|} \hat{y}_t$$

- For weighted:

$$resp = \sum_{t=1}^{|\mathcal{D}|} w_t \hat{y}_t$$



Ensemble Selection

- Conventional wisdom indicates as the number of base learners increases, accuracy and resistance to overfitting should improve.
- As the number of base learners increases, so does computational complexity.
- As the number of base learners increases, diversity can start to decrease, thus *reducing* the performance (accuracy) of the ensemble.
- This suggests there might be an “optimal” subset of base learners to be used in a given ensemble.



Ensemble Selection

- Selecting the base learners to make up an ensemble is a “subset selection” problem.
- Approaches to subset selection are also applied in “feature” selection (addressed in the module on dimensionality reduction).
- Common approaches:
 - Stepwise selection:
 - Forward
 - Backward
 - Local search
 - Wrapper-based
 - Filter-based



Example: Forward Model Selection

A wrapper-based approach:

1. Begin with an empty ensemble
2. Select the base learner from a library of base learners
 - a. Approach 1: based on individual model performance on a validation set
 - b. Approach 2: based on maximizing ensemble performance on a validation set
3. Repeat step 2 as long as ensemble performance continues to improve, or until all models in the library have been examined
4. Return the selected subset of based learners

Assumes the models can be ordered based on performance, but still requires evaluation of the ensemble as it is generated; this is what makes it a wrapper method.

Bootstrap Aggregation

- Perhaps one of the most basic methods for constructing ensembles.
- Has strong statistical properties from which uncertainty quantification can be performed.
- Referred to as “bagging” for short.
- Results in a set of “diverse” base learners due to the fact they are trained on different subsets of the training data.



The Bagging Process

- Generate b different bootstrap training sets, $\mathbf{X}_1, \dots, \mathbf{X}_b$, each of size $|\mathbf{X}|$.
- Each data set $\mathbf{X}_i$ is generated by sampling from $\mathbf{X}$ *with replacement* until $|\mathbf{X}|$ examples are selected.
- Train b different base learners using the appropriate data set. In other words, train base learner $\mathcal{D}_i$ using $\mathbf{X}_i$ as the training set.
- The combiner is (weighted) majority or (weighted) mean/median.
- **REMARK:** Bagging is optimized when the base learners exhibit high variance and low bias, such as a decision tree.



Random Forests

- Breiman defines a random forest as follows.

Def: A *random forest* is a classifier consisting of a collection of tree-structured classifiers $\{h(\mathbf{x}, \Theta_k), k = 1, \dots, m\}$ where $\{\Theta_k\}$ are independent, identically distributed random vectors and each tree casts a unit vote for the most popular class at input $\mathbf{x}$.

- This can be adapted for regression using averaging instead of voting.
- A common variant on this method is to also select a random subset of features from which to generate the trees.
- **REMARK:** Several cases have been demonstrated where random forests show competitive performance on tasks often “solved” via deep learning.



Random Forests

Algorithm 12.3 The Random Forest Algorithm

```
1: function RANDOMFOREST( $\mathcal{D}$ ,  $k$ ,  $n$ )
2:   //  $\mathcal{D}$  is the training set.
3:   //  $k$  is the number of available attributes
4:   //  $n$  is the number of classifiers to be constructed
5:    $\mathcal{L} \leftarrow \emptyset$ 
6:   for  $i \leftarrow 1$  to  $n$  do
7:      $\mathcal{D}_i \leftarrow \text{RandSubset}(\mathcal{D}, k)$ 
8:      $\ell_i \leftarrow \text{DTSubsetLearn}(\mathcal{D}_i)$ 
9:      $\mathcal{L} \leftarrow \mathcal{L} \cup \ell_i$ 
10:  end for
11:  return  $\mathcal{L}$ 
12: end function
```



Properties of Random Forests

- The trees in a RF are simpler and therefore easier to construct.
- RF ensembles have been shown to be highly accurate and resistant to overfitting (ensembles naturally regularize).
- Individual trees are considered to be relatively easy to interpret (although, full ensembles are not).
- RF ensembles provide a level of interpretability through the ability to do variable importance analysis.
- Technically, RF ensembles do not need cross-validation – can use out-of-bag samples for generalization assessment. Often, we still cross-validate.
- RF ensembles can also be used for clustering by counting the number of times pairs of points appear in the same leaves across the set of trees.
- There exist many variants.



Possible Variants

- Individual trees have all features available and grow trees with optimal splits fully on bootstrapped samples.
- Individual trees have a random subset of features available and grow trees with optimal splits on bootstrapped samples.
- Individual trees are grown with random splits.
- Only “stumps” are used as base learners where each stump is generated based on information gain.
- Only “random stumps” are used as base learners.
- Oblique (multi-variate) splits are allowed when generating the trees.
- A “spatial” variant considers spatial correlation in selecting features.



Boosting

- Invented independently by Freund (1990) and Shapire (1990).
- A sequential method of constructing ensembles by re-weighting the training data based on prior performance.
- As a sequential (iterative) process, stops generating base learners either when a target number (tunable) are generated or when performance of the ensemble has plateaued.
- Assumes weak learners, consisting of high bias and low variance.
- Tends to work poorly with noisy data since it tends to over-weight the noisy examples.



Basic Boosting

The “original” boosting algorithm only built an ensemble of three base learners.

1. For training set $\mathbf{X}$, where $|\mathbf{X}| = n$, sample $n_1 < n$ examples $\mathbf{X}_1 \subset \mathbf{X}$ without replacement.
2. Train base learner ℓ_1 on this sample.
3. Sample a second set of $n_2 < n$ examples $\mathbf{X}_2 \subset \mathbf{X}$ such that
 - a. The first $n_2/2$ examples correspond to the misclassified examples from $\mathbf{X}_1$ when using ℓ_1 .
 - b. The remaining $n_2/2$ are drawn from $\mathbf{X}$ without replacement but correctly classified by ℓ_1 .
4. Train based learner ℓ_2 from $\mathbf{X}_2$
5. Apply ℓ_1 and ℓ_2 to $\mathbf{X}$ and extract all examples where they disagree to yield $\mathbf{X}_3 \subset \mathbf{X}$.
6. Train ℓ_3 on $\mathbf{X}_3$
7. Use ensemble via majority vote.



Adaptive Boosting

Algorithm 12.2 Discrete AdaBoost Algorithm

```
1: function ADABOOST( $\mathcal{D}$ ,  $\mathcal{L}_{weak}$ ,  $L$ )
2:   //  $\mathcal{D}$  is the training set.
3:   //  $\mathcal{L}_{weak}$  is a weak learning algorithm
4:   //  $L$  is the number of classifiers to be constructed
5:    $\mathbf{w}^1 \leftarrow [1 \cdots 1]$   $\triangleright$   $\mathbf{w}$  has  $1/|\mathcal{D}|$  component values; the distribution is uniform.
6:   for  $i \leftarrow 1, L$  do
7:      $\mathbf{p}^i \leftarrow \frac{\mathbf{w}^i}{\sum_{j=1}^{|\mathcal{D}|} w_j^i}$   $\triangleright$  Derives sampling distribution from weights
8:      $\ell_i \leftarrow \text{Train}(\mathcal{L}_{weak}, \mathcal{D}, \mathbf{p}^i)$   $\triangleright$  Train after sampling using  $\mathbf{p}^i$ 
9:      $error_i \leftarrow \sum_{j=1}^{|\mathcal{D}|} p_j^i |\ell_i(\mathbf{x}_j) - y_j|$ 
10:     $\beta_i = error_i / (1 - error_i)$ 
11:    for  $j \leftarrow 1, |\mathcal{D}|$  do
12:       $w_j^{i+1} \leftarrow w_j^i \beta_i^{1 - |\ell_i(\mathbf{x}_j) - y_j|}$ 
13:    end for
14:  end for
15:  return  $\ell_1, \dots, \ell_L, \beta_1, \dots, \beta_L$ 
16: end function
```

- Often referred to as “AdaBoost.”
- Often uses decision trees (or decision stumps) as base learners, but this is not required.
- Algorithm to the left is also referred to as the AdaBoost.M1 algorithm.
- Choose class as

$$class = \operatorname{argmax}_c \sum_{i=1}^L \left(\log \frac{1}{\beta_i} \right) \ell_i^c(\mathbf{x}_j)$$



Advanced Boosting

- More advanced boosting algorithms have been developed in recent years.
- Examples of prominent algorithms include:
 - **Gradient Boosting** – Train learners correlated with negative gradient of the loss function with respect to the entire ensemble.
 - **XGBoost** – Gradient boosting with regularization added to the loss function.
 - **LightGBM** – Gradient boosting with one-sided sampling (reduces training examples needed) and feature bundling (reduces features to be evaluated).
 - **CatBoost** – Omits the sample being used to train when estimating the gradient, similar to a “jackknife” sample.
- To focus on common characteristics, we will discuss basic gradient boosting.



Gradient Boosting

- Core Idea: Develop base learners/models that are correlated highly with the gradient of the loss function for the entire ensemble.
- The goal is to learn an approximation $\hat{F}(\mathbf{x})$ of an underlying target function $F(\mathbf{x})$ by minimizing loss function $\mathcal{L}(y, F(\mathbf{x}))$ and constructing a weighted sum of functions.

$$\hat{F}_m(\mathbf{x}) = \hat{F}_{m-1}(\mathbf{x}) + \rho_m f_m(\mathbf{x})$$

where ρ_m is the weight of the m th function, $f_m(\mathbf{x})$. Here, we assume that $\hat{F}_0(x) = \operatorname{argmin}_f \mathcal{L}(y, f(\mathbf{x}))$.

- Each successive base learner finds $\rho_m f_m(\mathbf{x}) = \operatorname{argmin}_{\rho, f} \sum_{i=1}^n \mathcal{L}(y_i, \hat{F}_{m-1}(\mathbf{x}_i) + \rho f(\mathbf{x}_i))$.
- Significantly, the first learner trains on the original data, and subsequent learners train on the *residuals* of the ensemble so far.

$$r_{mi} = \left. \frac{\partial \mathcal{L}(y_i, \hat{F}(\mathbf{x}))}{\partial \hat{F}(\mathbf{x})} \right|_{\hat{F}(\mathbf{x}) = \hat{F}_{m-1}(\mathbf{x})} .$$



Gradient Boosting

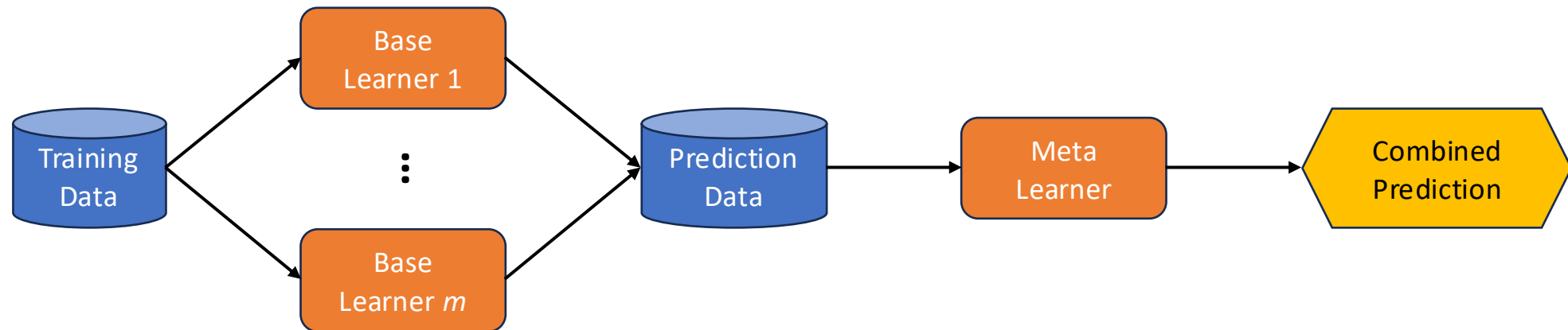
Algorithm 1.1 Gradient Boosting

```
1: function GRADBOOST( $\mathbf{X}, \mathcal{L}, T$ )
2:   Initialize with  $\hat{F}_0(\mathbf{X}) = \operatorname{argmin}_f \sum_{\mathbf{x}_i \in \mathbf{X}} \mathcal{L}(y_i, f(\mathbf{x}_i))$ 
3:   for  $m = 1, \dots, M$  do
4:     for  $\mathbf{x}_i \in \mathbf{X}$  do
5:       Calculate  $r_{mi} = \left. \frac{\partial \mathcal{L}(y_i, \hat{F}(\mathbf{x}_i))}{\partial \hat{F}(\mathbf{x}_i)} \right|_{\hat{F}(\mathbf{X}) = \hat{F}_{m-1}(\mathbf{X})}$ 
6:     end for
7:     Train  $f_m(\mathbf{X})$  using  $\mathcal{D}_m = \{\mathbf{x}_i, r_{mi}\}_{\mathbf{x}_i \in \mathbf{X}}$ 
8:      $\rho_m f_m(\mathbf{X}) = \operatorname{argmin}_{\rho, f} \sum_{\mathbf{x}_i \in \mathbf{X}} \mathcal{L}(y_i, \hat{F}_{m-1}(\mathbf{x}_i) + \rho f(\mathbf{x}_i))$ 
9:      $\hat{F}_m(\mathbf{X}) = \hat{F}_{m-1}(\mathbf{X}) + \rho_m f_m(\mathbf{X})$ 
10:  end for
11:  return  $\hat{F}_M(\mathbf{X})$ 
12: end function
```



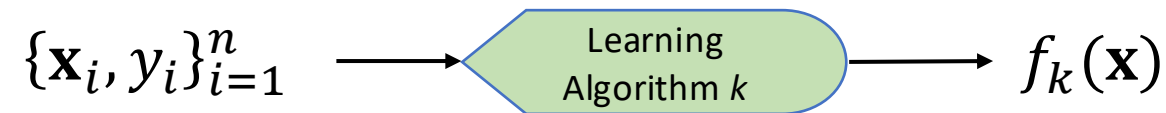
Stacking Introduction

- So far, all of the methods examined have assumed the models in the ensemble are of the same type.
- Stacking is a method that allows different types of models to be combined.
- The primary advantage is that stacking allows models with different inductive biases to contribute to the overall prediction based on their individual strengths.
- In stacking, a meta-learner is used to combine the predictions.

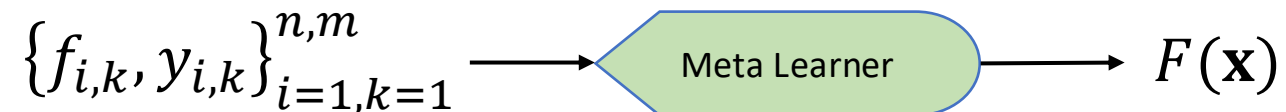


Learning for Stacking

- Based learners are referred to as “Level 0” models.
- The meta-learner result in a “Level 1” model.
- The purpose of the Level 1 model is to combine predictions.
- The data set for the Level 1 model consists of (predictions, target) pairs.
- Level 0:



- Level 1:



Stacking

Algorithm 1.1 Stacking

```
1: function STACK( $\mathbf{X}, T$ )
2:   for  $k = 1, \dots, m$  do
3:     Train  $f_k(\mathbf{X})$  using  $\mathbf{X}$ 
4:   end for
5:    $\mathbf{H} = \emptyset$ 
6:   for  $k = 1, \dots, m$  do
7:      $\mathbf{f}(\mathbf{x}_i) = []$ 
8:     for  $i = 1, \dots, |\mathbf{X}|$  do
9:        $\mathbf{f}(\mathbf{x}_i) \leftarrow \mathbf{f}(\mathbf{x}_i) || f_k(\mathbf{x}_i)$ 
10:    end for
11:     $\mathbf{H} \leftarrow \mathbf{H} \cup \{(\mathbf{f}(\mathbf{x}_i), y_i)\}$ 
12:  end for
13:  Train  $F(\mathbf{X})$  using  $\mathbf{H}$ 
14:  return  $f_1(\mathbf{X}), \dots, f_m(\mathbf{X}), F(\mathbf{X})$ 
15: end function
```



Types of Models

- Different types of models are used for the level 0 models (e.g., SVM, decision tree, neural network).
- For classification problems, the meta-model can be anything but is often logistic regression.
- If linear classifier insufficient, can move to a more sophisticated model.
- For regression problems, the meta-model can be anything but is often linear regression.
- Similar to classification, if linear regression insufficient, can move to a more sophisticated model.

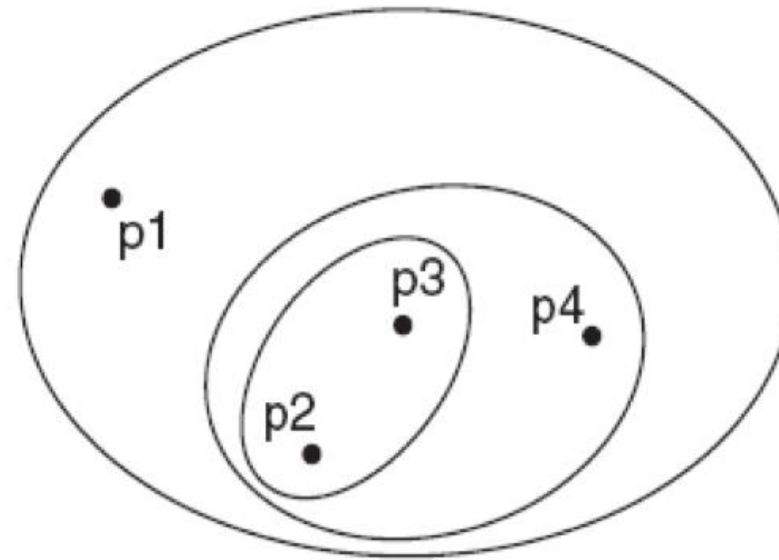
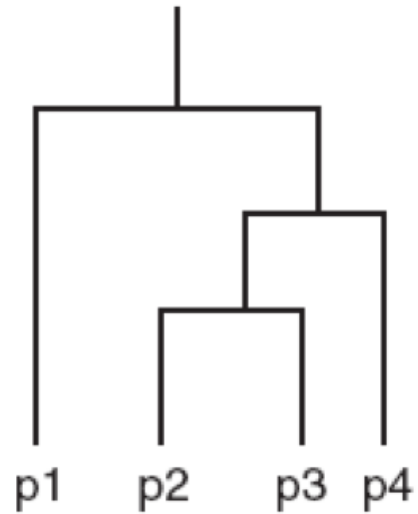


Overfitting

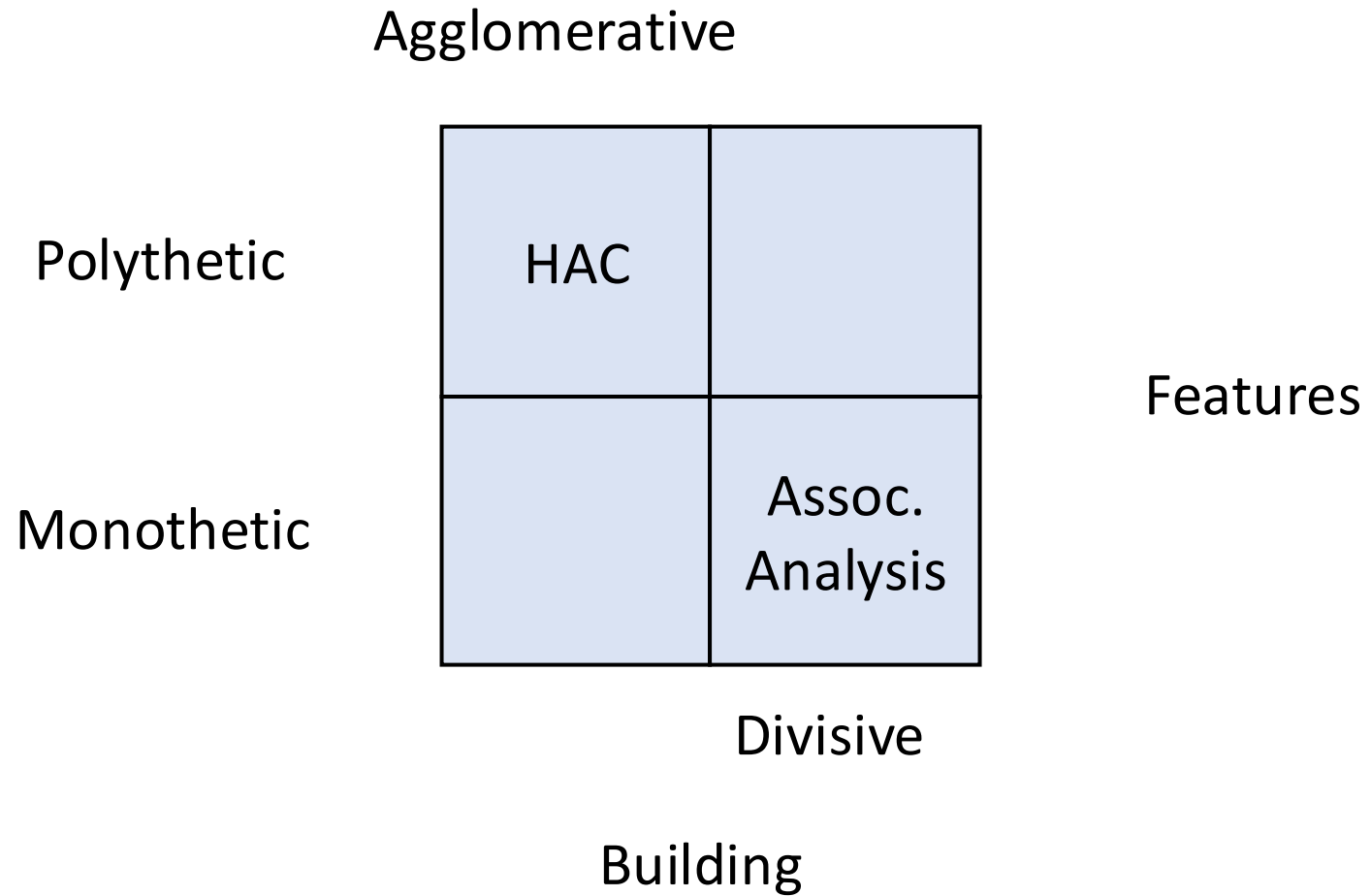
- The conventional wisdom is that all ensemble methods are robust with respect to overfitting.
- Regularization is based upon diversity among members of the ensemble.
- Bagging – Diversity comes from the individual data sets.
- Boosting – Diversity comes from re-weighting the data and then weighting the base learners.
- Gradient Boosting – Diversity comes from focusing on combining predictions focused on the residuals.
- Stacking – Diversity comes from the different inductive biases of the base learners.



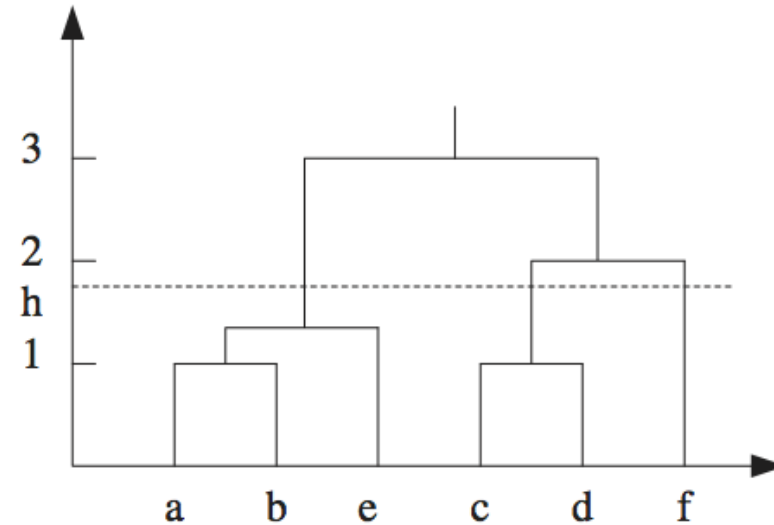
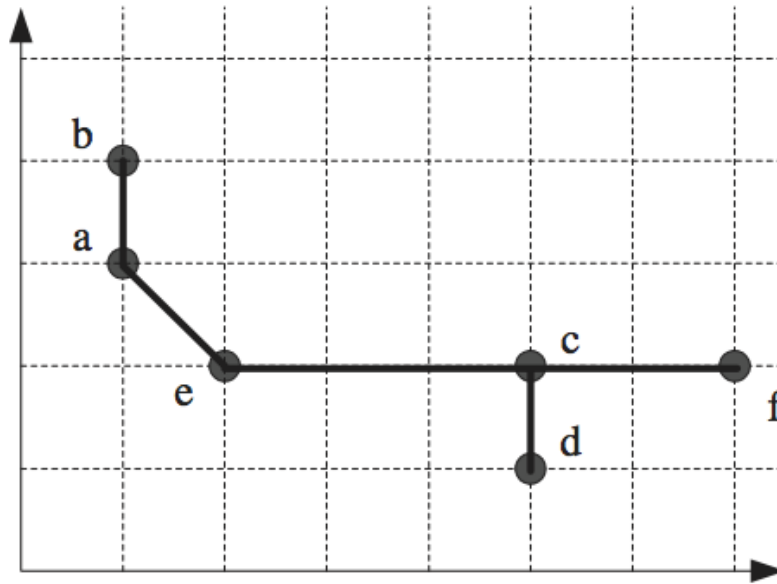
Dendrogram



Agglomerative vs Divisive



Hierarchical Agglomerative Clustering



Linkage Rules

- Single Linkage: Merge clusters such that

$$D(\mathbf{c}_i, \mathbf{c}_j) = \min_{\mathbf{x} \in \mathbf{c}_i, \mathbf{y} \in \mathbf{c}_j} \delta(\mathbf{x}, \mathbf{y})$$

- Complete Linkage: Merge clusters such that

$$D(\mathbf{c}_i, \mathbf{c}_j) = \min \left\{ \max_{\mathbf{x} \in \mathbf{c}_i, \mathbf{y} \in \mathbf{c}_j} \delta(\mathbf{x}, \mathbf{y}) \right\}$$

- Average Linkage: Merge clusters such that

$$D(\mathbf{c}_i, \mathbf{c}_j) = \min \left\{ \frac{1}{m_1 m_2} \sum_{\mathbf{x} \in \mathbf{c}_i, \mathbf{y} \in \mathbf{c}_j} \delta(\mathbf{x}, \mathbf{y}) \right\}$$

- Ward's Method: Merge clusters such that

$$D(\mathbf{c}_i, \mathbf{c}_j) = \min \frac{TSS - ESS}{ESS}, ESS = \sum_i \sum_j \sum_k |x_{ijk} - \bar{x}_{i..k}|^2, TSS = \sum_i \sum_j \sum_k |x_{ijk} - \bar{x}_{...k}|^2$$



Association Analysis

- Recall

$$\chi^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i}$$

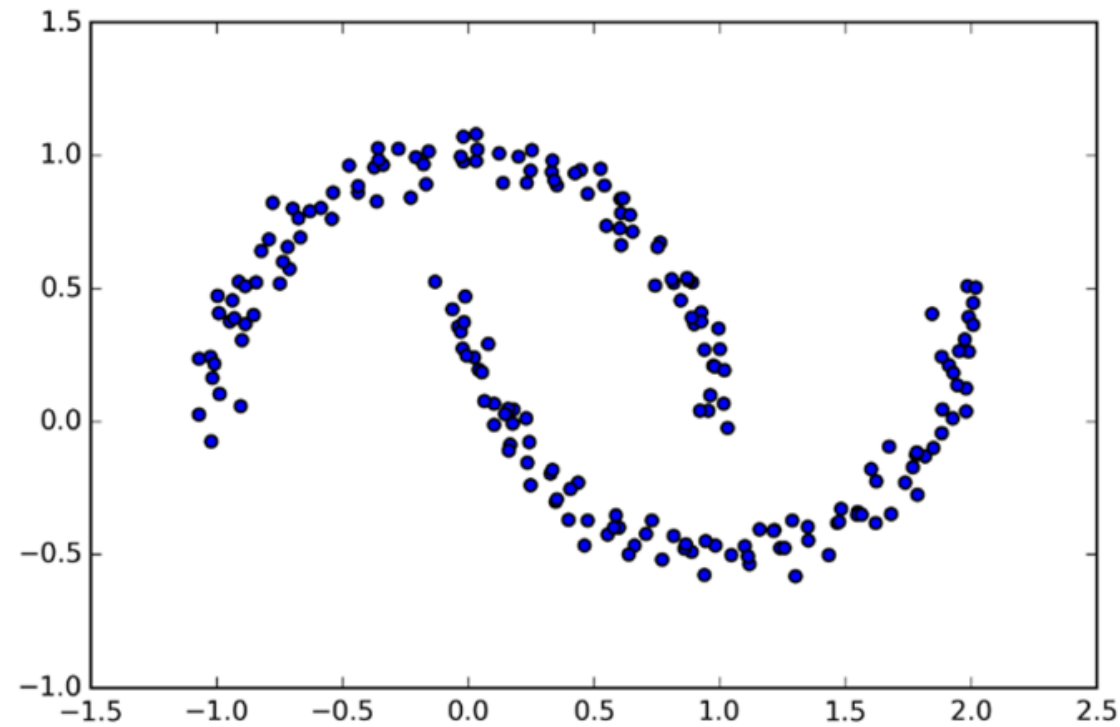
- Binary $\rightarrow$ Normalized
- F_{ij} is the j^{th} attribute of the i^{th} data point
- F_{ik} is the k^{th} attribute of the i^{th} data point

$$\begin{aligned} a_{jk} &= \sum_{x_i} F_{ij} \times F_{ik} \\ b_{jk} &= \sum_{x_i} (1 - F_{ij}) \times F_{ik} \\ c_{jk} &= \sum_{x_i} F_{ij} \times (1 - F_{ik}) \\ d_{jk} &= \sum_{x_i} (1 - F_{ij}) \times (1 - F_{ik}) \\ \chi_{jk}^2 &= \frac{(ad - bc)^2}{(a - b)(a - c)(b - d)(c - d)} \end{aligned}$$

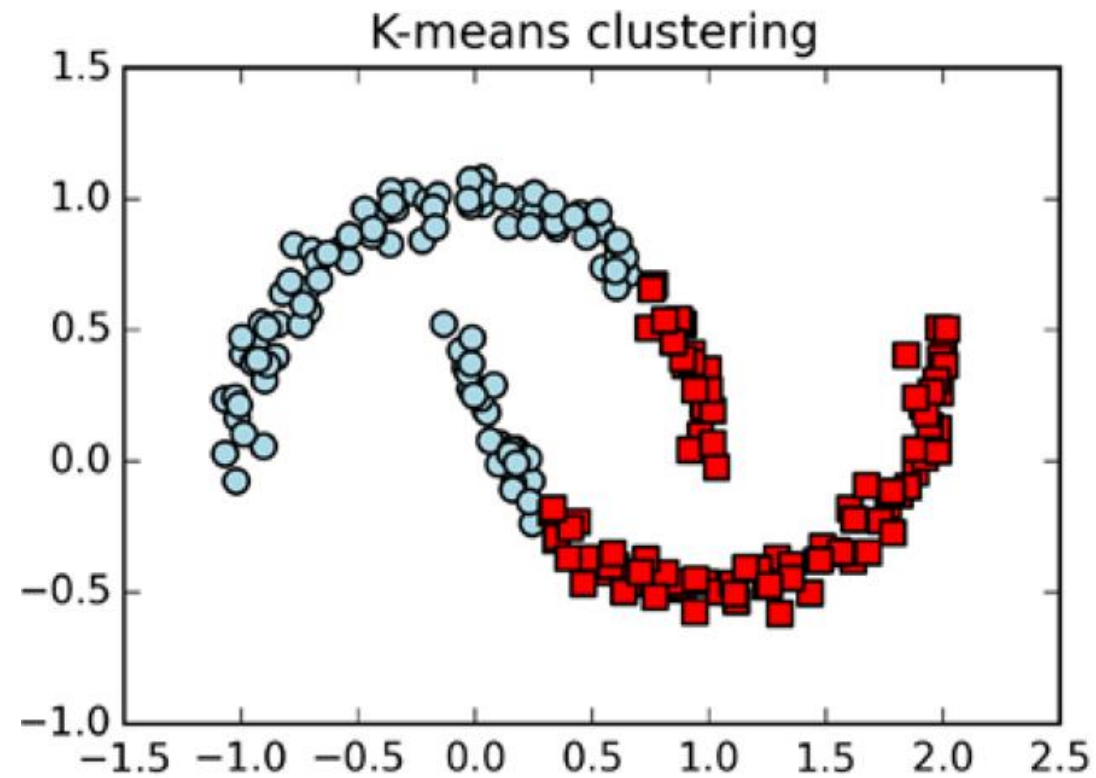
$$F_{split} = \operatorname{argmax} \sum_{k=1}^d \chi_{jk}^2$$



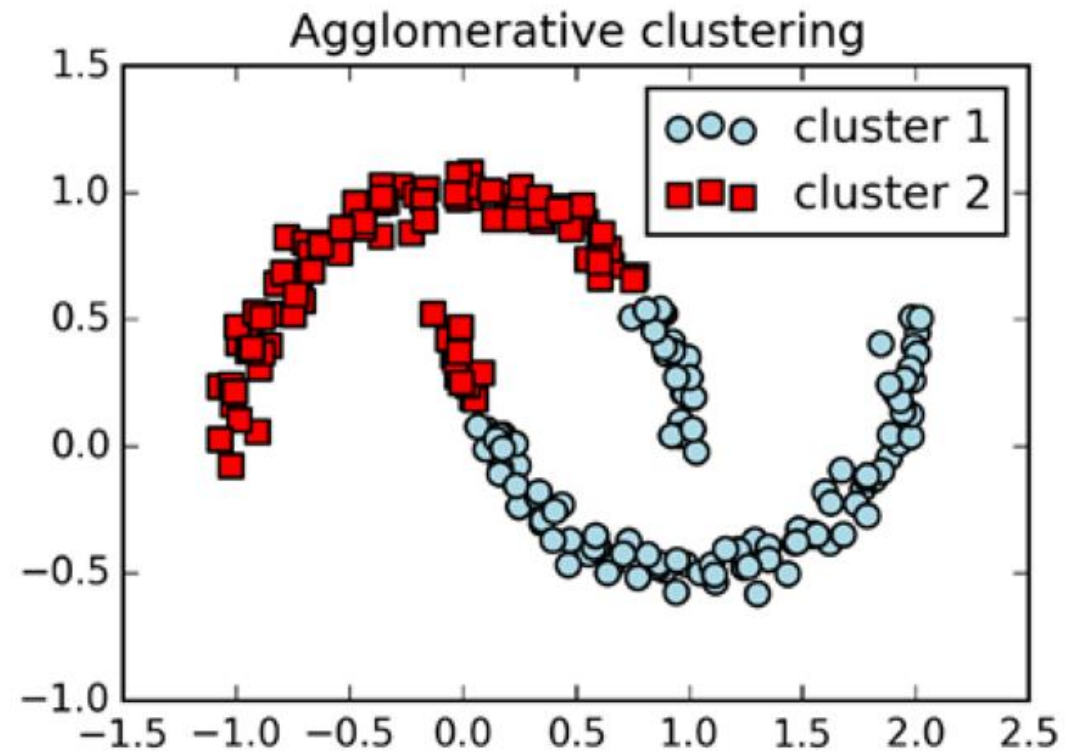
A Motivating Example



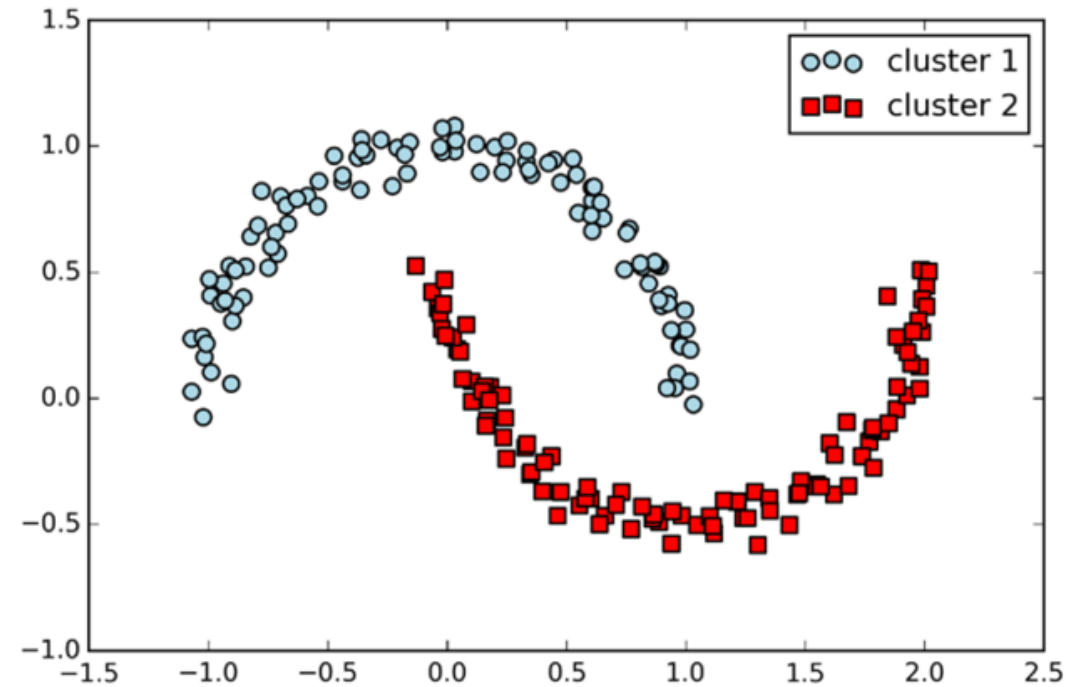
K-Means



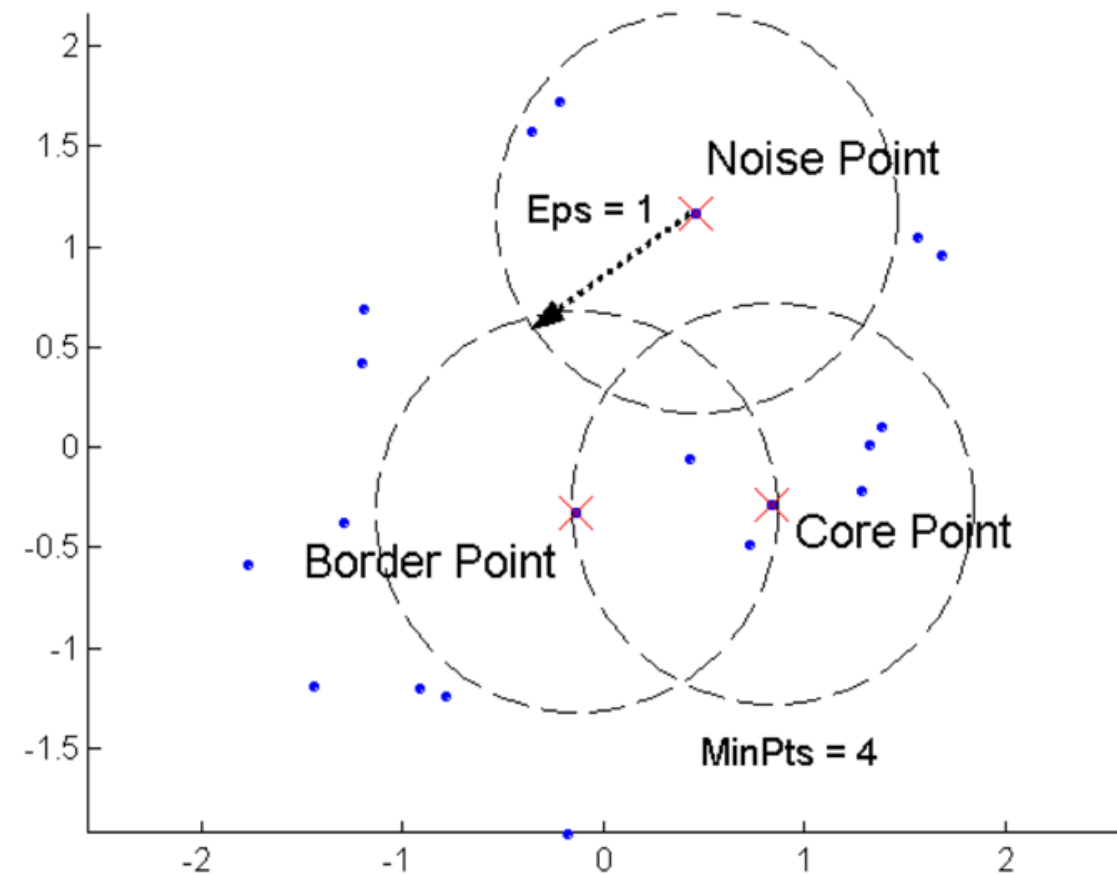
HAC



Density-based Clustering



Point Categorization



DBSCAN

Algorithm 2 DB-Scan

```
1: function DB-SCAN( $\mathcal{D}$ )
2:    $currClustLbl \leftarrow 1$ 
3:   for all  $p \in Core$  do do
4:     if  $clustLbl[p] = \text{“Unknown”}$  then
5:        $currClustLbl \leftarrow currClustLbl + 1$ 
6:        $clustLbl[p] \leftarrow currClustLbl$ 
7:     end if
8:     for all  $p' \in \theta\text{-neighborhood}$  do
9:       if  $clustLbl[p'] = \text{“Unknown”}$  then
10:         $clustLbl[p'] \leftarrow currClustLbl$ 
11:      end if
12:    end for
13:  end for
14:  return  $clustLbl$ 
15: end function
```



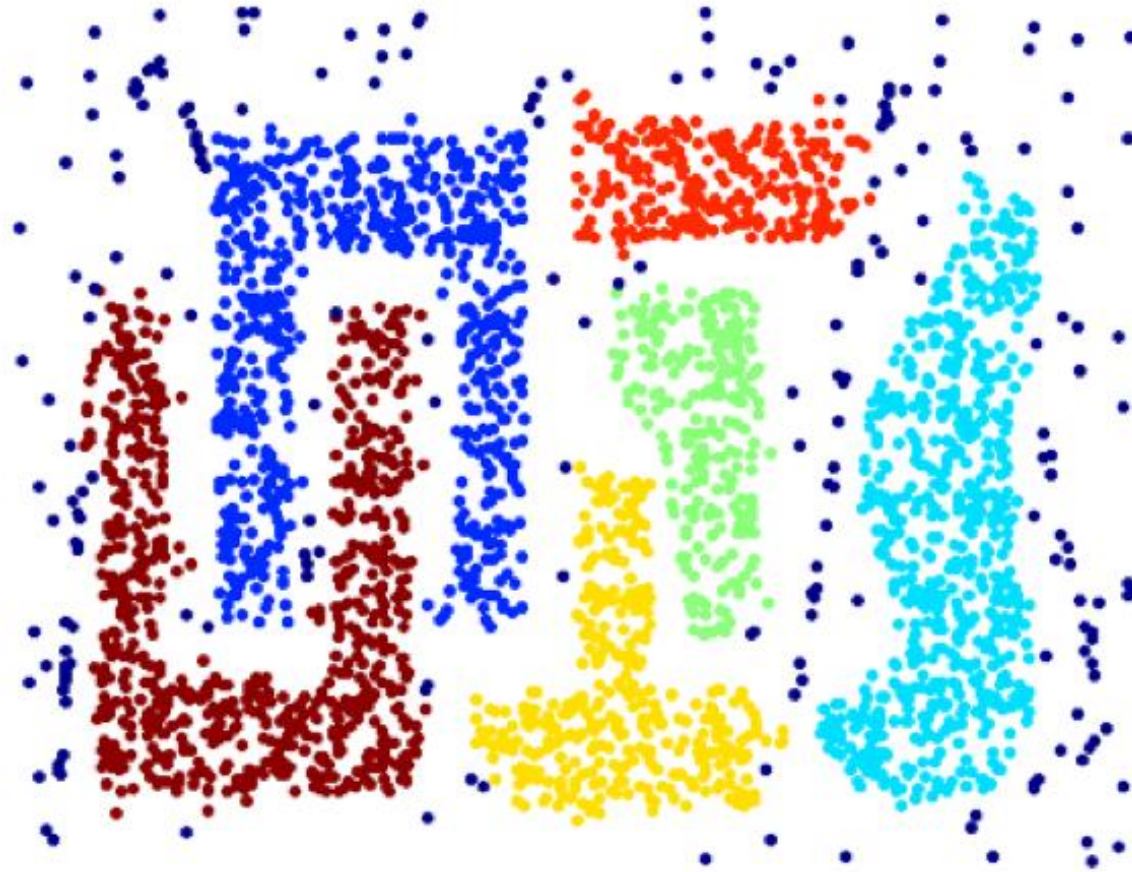
Another Example



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING

Another Example



MONTANA
STATE UNIVERSITY

NORM ASBJORNSON
College of
ENGINEERING